Methods in Java

27 October 2026 20:23

What is a Method in Java?

A method is a block of code, or a collection of statements, or a set of code grouped to perform a certain task or operation.
It is used to achieve the reusability of code. We write a method once and use it many times. We do not need to write code again and again. It also provide easy modification and reliability of code just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

The most important method in Java is the **main()** method.

Advantage of using methods

The following are the advantages of using methods in Java:

- **Reusability**: Methods provide code reusability. Once a method is defined, it can be called multiple times from various parts of the program.
- **Modularity**: Methods help breakdown complex problems into smaller, manageable pieces.
- **Maintainability**: With methods, updating and debugging code become easiest, since changes in Word part of the program do not necessarily impact other parts.
- **Abstraction**: Methods provide a way to encapsulate implementation details, exposing only the necessary parts to the users.

Method naming rules

While defining a method, remember that the method name must be a verb and start with a lowercase letter. If the method name has more than two words, the first name must be verb followed by an adjective or noun. In the multi - word method name, the first letter of each word must be in uppercase except for the first word. For example:

Single-word method name: sum(), area()

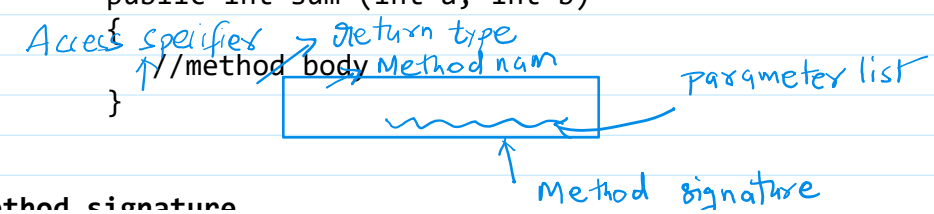
Multi-word Method Name: areaOfCircle(), stringComaprison()

It is also possible that the method has the same name as another method in the same class. It is known as method overloading.

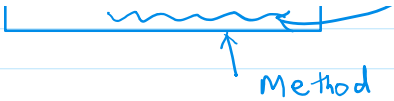
Method declaration

The method declaration provides information about method attributes such as visibility, return type, name, and arguments. It has six components that are known as method header.

```
public int sum (int a, int b)
{
  //method body
}
```



Method signature



Method signature

Every method has a method signature. It is a part of method declaration. It includes the method name and parameter list.

Access Specifier

Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifiers:

- **public:** The method is accessible to all classes when we use the public specifier in our application.
- **private:** When we use a private access specifier, the method is accessible only in the class in which it is defined.
- **protected:** When we use a protected access specifier, the method is accessible within the same package or subclasses in a different package.
- **default**: When we do not use any access specifier in the method declaration, Java uses the default access specifier by default. It is visible only from the same package.

Return type

Return type is the data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use the void keyword.

Method Name

It is a unique name that is used to define the name of a method. It must correspond to the functionality of the method. Suppose if we are creating a method for subtraction of two numbers, the method name must be subtraction(). A method is invoked by its name.

Parameter list

It is a list of parameters separated by comma and enclosed in a pair of parentheses. It contains the data type and variable name. If the method has no parameter, leave the parentheses blank.

Method body

It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within a pair of curly braces.

The method body is enclosed in curly braces {} and contains the statements that define the functionality or working of the method. For example, consider the following code snippet:

```
public class Example1
{
    //Method declaration
    public int add(int a, int b){
        int sum = a+b; //method body
```

```
            return sum; //return statement
        }
    }
```

Types of Methods

There are two types of methods in Java:

- Predefined Method
- User-defined Method

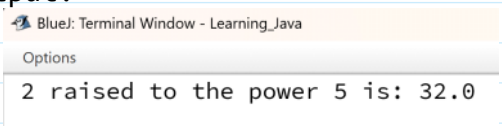
Predefined Method

The method that are already defined in Java class libraries are known as predefined methods. It is also known as standard library method or build - in method. We can directly use this method just by calling them in a program at any point.

```
package Methods;


/**
 * Write a description of class Example1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Example1
{
    public static void main(String[] args){
        System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
    }
}
```

Output:



User-defined Method

The method written by the user or programmer is known as a user defined method. These methods can be modified or customized according to the requirements.

Calling User defined Method

In Java, calling or invoking a method depends on whether the method is static or non-static. If the method is static, we can call it directly using the class name or from within the same class. Note that we need to create an object for calling non-static methods.

Syntax:

```
static return-type method-name(<parameter-list>){
   //code
}

ClassName.method-name(arg1,arg2,arg3,…);
```

If the method is non static, we must create an object of the class and use it to call the method.

```
10 public class Example1
11 {
12     //Method declaration
13     public int add(int a, int b){
14         int sum = a+b; //method body
15         return sum; //return statement
16     }
17
18     public static void main(String[] args){
19         System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
20         add(10, 15);
21     }
22 }
```

non-static method add(int,int) cannot be referenced from a static context

Error happens, because we haven't created object Example1 class, and calling the non-static method.

```
package Methods;


/**
 * Write a description of class Example1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Example1
{
    //Method declaration
    public int add(int a, int b){
        int sum = a+b; //method body
        return sum; //return statement
    }

    public static void main(String[] args){
        System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
        Example1 ob = new Example1(); //created object of Example1 class
        int result = ob.add(10, 15); //calling a non-static method using the
object of the class
        System.out.print("\nsum = " + result);
    }
}
```

Output:

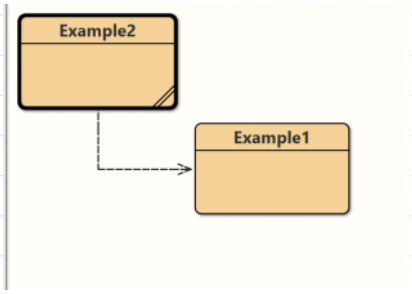
```
BlueJ: Terminal Window - Learning_Java
Options
 2 raised to the power 5 is: 32.0
sum = 25
```

```
package Methods;
```

```
/**
 * Write a description of class Example1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Example1
{
    //Method declaration
    public static int add(int a, int b){
        int sum = a+b; //method body
        return sum; //return statement
    }

    public static void main(String[] args){
        System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
        Example1 ob = new Example1(); //created object of Example1 class
        // int result = ob.add(10, 15); //calling a non-static method using the
object of the class
        int result =  add(2,5);
        System.out.print("\nsum = " + result);
    }
}
```

calling a static function.



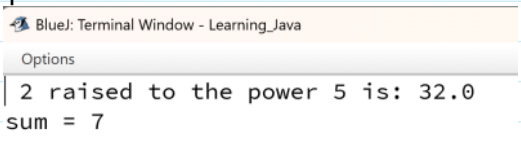

```
public class Example2
{

    public static void main(String[] args){
        System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
        Example2 ob = new Example2(); //created object of Example2 class
        // int result = ob.add(10, 15); //calling a non-static method using the
object of the class
        int result =  Example1.add(2,5);
        System.out.print("\nsum = " + result);
    }
}
```

class reference to call static method from that class.

If the static method is present in the same class, then we can call it directly, without any reference.

Output:



```
package Methods;


/**
 * Write a description of class Example1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Example1
{
    //Method declaration
    public static int add(int a, int b){
        int sum = a+b; //method body
        return sum; //return statement
    }

    public void displaySum(int x, int y){
        System.out.print("\nSum = " + add(x, y));
    }

    public static void main(String[] args){
        System.out.print("\f 2 raised to the power 5 is: " + Math.pow(2, 5));
        Example1 ob = new Example1(); //created object of Example1 class
        // int result = ob.add(10, 15); //calling a non-static method using the
object of the class
        int result =  add(2,5);
        System.out.print("\nsum = " + result);

        ob.displaySum(3, 7);
    }
}
```

static-method called directly from a non-static method of a class.

```
10 public class Example1
11 {
12     //Method declaration
13     public static int add(int a, int b){
14         int sum = a+b; //method body
15         displaySum(a, b);
16         return
17     }
18
19     public void displaySum(int x, int y){
20         System.out.print("\nSum = " + add(x, y));
21     }
```

non-static method displaySum(int,int) cannot be referenced from a static context

from static method we cannot call non-static method.

Note: We will get a compile-time error if we try to call a non-static method from a static context without an object. Because non-static method belongs to instance not the class itself.